MINECRAFT PATHFINDING LOGIC AND APPLICATIONS

By Fox3000foxy

SUMMARY

INTRODUCTION 3
HOW MOBS AI WORKS 4
GOALS 4
MOUVEMENT 5
MOB PREFERENCES 10
PATHFINDING ALGORITHM 12
PATHFINDING EXPLOITS 14
How A* Works in Minecraft 14
Two Major Quirks That Make A* Abusable 16
Example: Choosing Between Two Paths 19
EXERCISE 20
Different Mobs Choose Different Paths 23
How to Assign Targets to Mobs 24
Recap: 28

INTRODUCTION

The path-finding logic of Minecraft mobs is governed by a set of rules that specify how each creature behaves based on its type and its surroundings. We'll watch how those laws are defined and how they can be exploited to design structures that will improve farms' efficiency, or just how we can mess around with those mobs.

This is one of the most effective pathfinding strategies I've discovered in Minecraft, and it can be applied to survival. Mobs can be directed into traps, farms can be optimized, or even invisible AI paths can be created by simply and covertly manipulating where they will find the path. I'll demonstrate how to take control of Minecraft's AI and manipulate it. You'll realize by the end that mob pathfinding isn't at all random. It is fully exploitable, score-based, and predictable.

HOW MOBS AI WORKS

Before you're ready to do all of that, though, I first need to explain how mob AI works:

GOALS

The AI for all mobs starts with AI goals. These tell us what the mob can do and how likely it is to do it. The one we're going to focus on today is **WaterAvoidingRandomStroll**. Note that it has a priority of seven. Lower numbers mean higher priority. Higher-priority behaviors can interrupt lower-priority behaviors:

```
protected void registerGoals() {
   this.goalSelector.addGoal(4, new Zombie.ZombieAttackTurtleEggGoal(this, 1.0, 3));
   this.goalSelector.addGoal(8, new LookAtPlayerGoal(this, Player.class, 8.0F));
   this.goalSelector.addGoal(8, new RandomLookAroundGoal(this));
   this.addBehaviourGoals();
}

protected void addBehaviourGoals() {
   this.goalSelector.addGoal(2, new ZombieAttackGoal(this, 1.0, false));
   this.goalSelector.addGoal(6, new MoveThroughVillageGoal(this, 1.0, true, 4, this::canBreakDoors));
   this.goalSelector.addGoal(7, new WaterAvoidingRandomStrollGoal(this, 1.0));
   this.targetSelector.addGoal(1, new HurtByTargetGoal(this).setAlertOthers(ZombifiedPiglin.class));
   this.targetSelector.addGoal(2, new NearestAttackableTargetGoal(this, Player.class, true));
   this.targetSelector.addGoal(3, new NearestAttackableTargetGoal(this, AbstractVillager.class,
false));
   this.targetSelector.addGoal(3, new NearestAttackableTargetGoal(this, IronGolem.class, true));
   this.targetSelector.addGoal(5, new NearestAttackableTargetGoal(this, Turtle.class, 10, true, false,
Turtle.BABY_ON_LAND_SELECTOR));
}
```

Here the ZombieAttackTurtleEggGoal is lower priority than ZombieAttackGoal so the zombie will prefer attack the villager instead of breaking the eggs.

MOUVEMENT

Every tick, a canUse function is called, which determines when it gets triggered. This occurs on a random interval which defaults to 1 in 60. If the 1-in-60 chance hits, the mob will wander; otherwise, it won't be this tick.

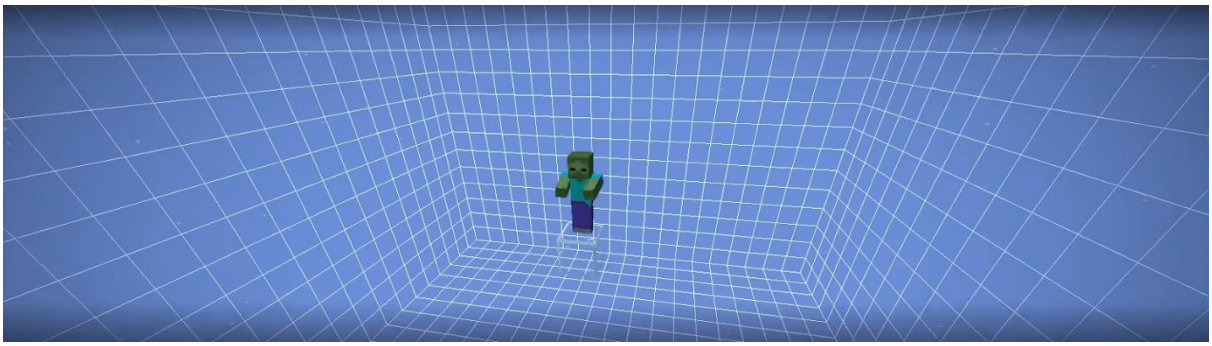
```
public boolean canUse() {
   if (this.mob.hasControllingPassenger()) {
      return false;
   } else {
      if (!this.forceTrigger) {
         if (this.checkNoActionTime && this.mob.getNoActionTime() >= 100) {
            return false;
         }

         if (this.mob.getRandom().nextInt(reducedTickDelay(this.interval)) != 0) {
            return false;
         }
      }

      Vec3 $$0 = this.getPosition();
      if ($$0 == null) {
         return false;
      } else {
         this.wantedX = $$0.x;
         this.wantedY = $$0.y;
         this.wantedZ = $$0.z;
         this.forceTrigger = false;
         return true;
      }
   }
}
```

Next, let's see what happens when the mob does wander. To determine where the mob will find to, the getPosition function is called. This first checks that the mob is within water. If this is not the case, then the code will randomly decide whether to use a

random land position or just a **random position**. The odds of the latter happening are quite literally 0.1%.

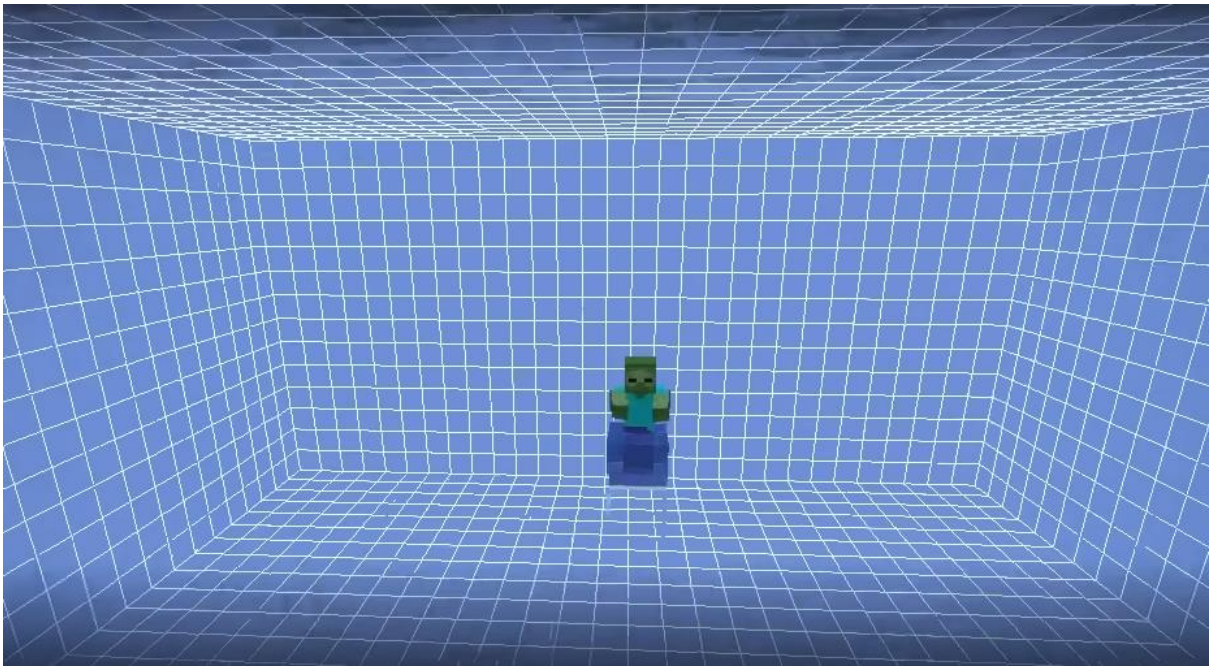
```
protected Vec3 getPosition() {
   if (this.mob.isInWater()) {
      Vec3 $$0 = LandRandomPos.getPos(this.mob, 15, 7);
      return $$0 == null ? super.getPosition() : $$0;
   } else {
      return this.mob.getRandom().nextFloat() >= this.probability ? LandRandomPos.getPos(this.mob, 10,
7) : super.getPosition();                                          1%
   }
}
```

It's helpful to note here that the two numbers at the end of this call represent the XZ radius and the Y radius respectively. This is the range in which the random point can be generated.



Radius from 10 large and 7 height 1

However, if the mob *is* in water, the game checks an even wider range. If it fails to find a path to land in that call, then it just defaults to a regular getPosition call, which can pathfind into water. This means that mobs prefer to walk **out of water**.



Radius from 15 large and 7 height 1

Let's step into LandRandomPos, because this is where things get weird.

```
public static Vec3 getPos(PathfinderMob $$0, int $$1, int $$2, ToDoubleFunction<BlockPos> $$3) {
   boolean $$4 = GoalUtils.mobRestricted($$0, $$1);
   return RandomPos.generateRandomPos(() -> {
      BlockPos $$4xx = RandomPos.generateRandomDirection($$0.getRandom(), $$1, $$2);
      BlockPos $$5 = generateRandomPosTowardDirection($$0, $$1, $$4, $$4xx);
      return $$5 == null ? null : movePosUpOutOfSolid($$0, $$5);
   }, $$3);
}

public static BlockPos movePosUpOutOfSolid(PathfinderMob $$0, BlockPos $$1) {
   $$1 = RandomPos.moveUpOutOfSolid($$1, $$0.level().getMaxY(), $$1x -> GoalUtils.isSolid($$0, $$1x));
   return !GoalUtils.isWater($$0, $$1) && !GoalUtils.hasMalus($$0, $$1) ? $$1 : null;
}
```

First, in getPos, the code checks if the mob is restricted. Being attached to a lead is one example of this. Next, the game generates candidate positions for the mob to find a path. And this is where things get kind of crazy.

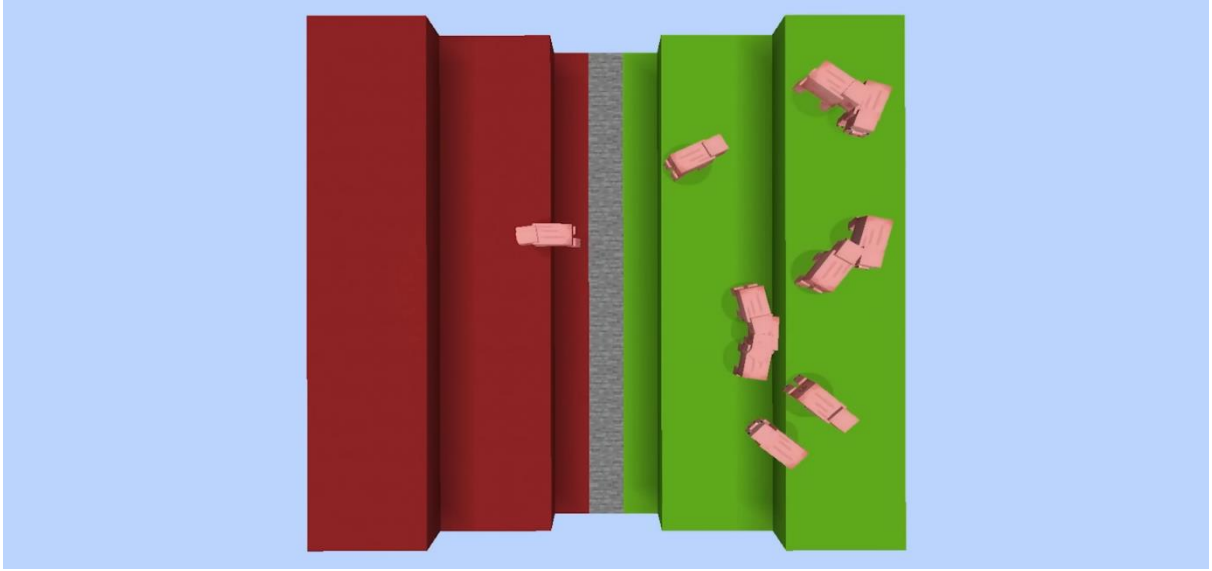
Some of the blocks chosen as candidates are inside of blocks. To remedy this problem - and to keep the process efficient - the code simply moves them up via movePosUpOutOfSolid.

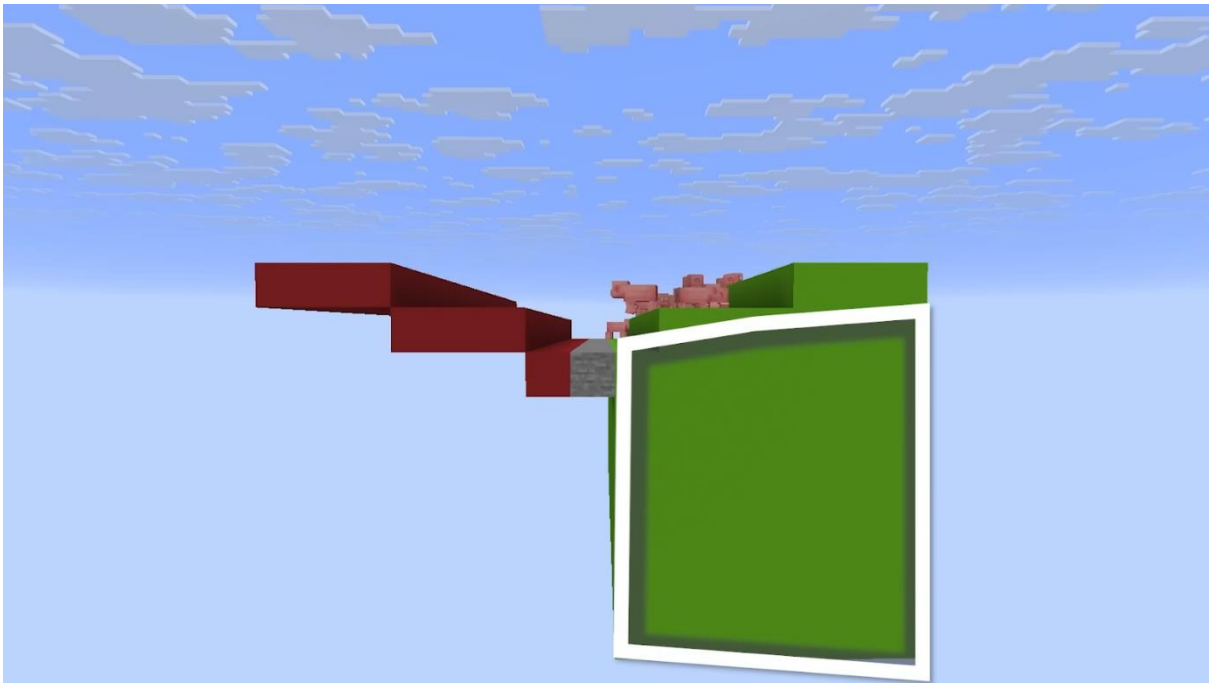
On the surface, this isn't a bad idea: it makes use of all those unused calls below ground, which improves performance. But this optimization also leads to some undesired results.

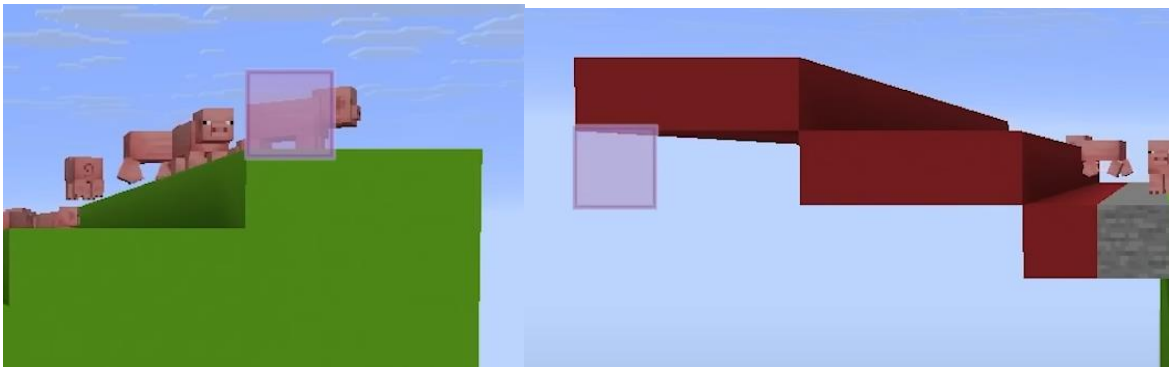
The most obvious: **mobs prefer to pathfind to high ground.**
This happens due to the higher density of blocks in high-ground areas. While this effect isn't desired, it is exploitable.

If we can recreate the conditions that cause mobs to prefer high ground, then we can control where mobs pathfind to.

Look at this setup:



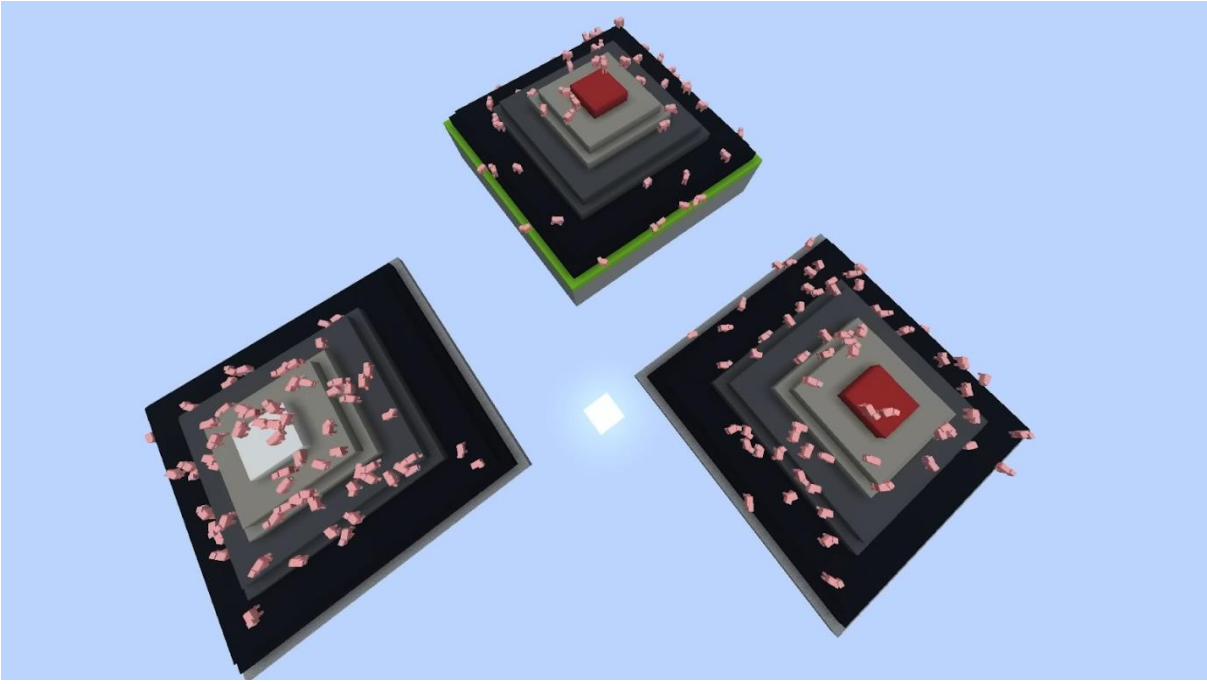




By placing these extra blocks on the right side, we give pigs more opportunities to pick positions from that side, biasing them heavily in that direction. This works because any positions chosen within the green area are moved up to the top of the green blocks by

the “move out of solid” function. In contrast, any positions chosen under the red simply fail because they’re in air.

This leads to the biased distribution you see.
But what can we do with it?
Look at these three pyramids.



From the top down, they’re identical - but from pig movement on top, we can tell the undersides are not.

- The bottom-left pyramid has all pigs’ pathfinding toward the middle.
- The top pyramid has pigs’ pathfinding toward the outside.
- The rightmost pyramid has pigs’ pathfinding randomly.

This is all explained by what’s underneath:

1. **First pyramid** - completely solid.
   The pigs prefer to pathfind to higher points since they have more blocks under them.

2. **Second pyramid** - completely hollow except the outside, which has a thick layer of blocks.
   This causes pigs to prefer the outside.

3. **Third pyramid** - completely hollow.
   This leads to a random distribution.

These are just simple examples of what you can do. And we can take it further.

Before a mob pathfinds a point, a final processing step chooses where it will **pathfind**.

```
public static Vec3 getPos(PathfinderMob $$0, int $$1, int $$2, ToDoubleFunction<BlockPos> $$3) {
   boolean $$4 = GoalUtils.mobRestricted($$0, $$1);
   return RandomPos.generateRandomPos(() -> {
      BlockPos $$4xx = RandomPos.generateRandomDirection($$0.getRandom(), $$1, $$2);
      BlockPos $$5 = generateRandomPosTowardDirection($$0, $$1, $$4, $$4xx);
      return $$5 == null ? null : movePosUpOutOfSolid($$0, $$5);
   }, $$3);
}
```

In this step, the 10 candidate positions generated earlier are scored. The highest scoring one wins.

```
public static Vec3 generateRandomPos(Supplier<BlockPos> $$0, ToDoubleFunction<BlockPos> $$1) {
   double $$2 = Double.NEGATIVE_INFINITY;
   BlockPos $$3 = null;

   for(int $$4 = 0; $$4 < 10; ++$$4) {
      BlockPos $$5 = (BlockPos)$$0.get();
      if ($$5 != null) {
         double $$6 = $$1.applyAsDouble($$5);
         if ($$6 > $$2) {
            $$2 = $$6;
            $$3 = $$5;
         }
      }
   }

   return $$3 != null ? Vec3.atBottomCenterOf($$3) : null;
}
```

This is neat because if we know what makes a block score higher, we can make mobs pathfind to it.

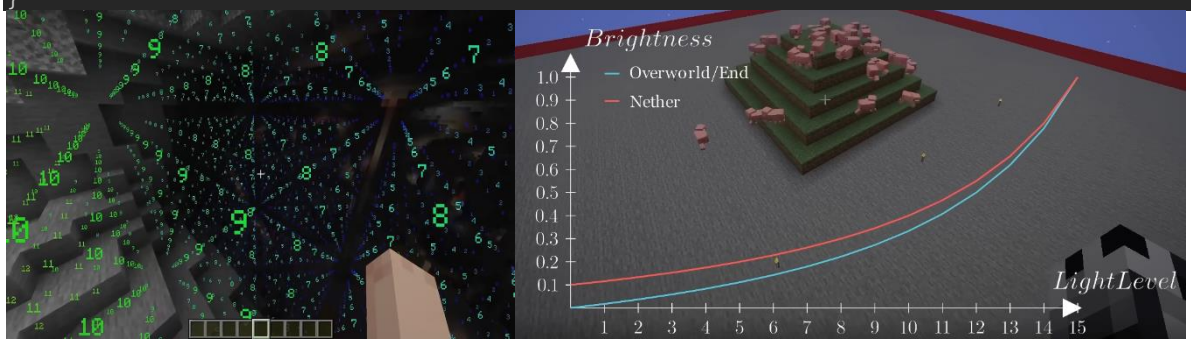
MOB PREFERENCES

Here's the quick rundown of mob preferences:

Animals prefer grass blocks, and well-lit blocks

Light level increases their path-finding score.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   return $$1.getBlockState($$0.below()).is(Blocks.GRASS_BLOCK) ? 10.0F :
$$1.getPathfindingCostFromLightLevels($$0);
}
```



If you're curious, here's the conversion from light level to pathfinding score.

Monsters prefer:

- darker places

They use the *inverse* of that light-score function.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   return -$$1.getPathfindingCostFromLightLevels($$0);
}
```

Turtles prefer water, then sand, and finally well-lit blocks

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   if (!this.goingHome && $$1.getFluidState($$0).is(FluidTags.WATER)) {
      return 10.0F;
   } else {
      return TurtleEggBlock.onSand($$1, $$0) ? 10.0F : $$1.getPathfindingCostFromLightLevels($$0);
   }
}
```

Hoglins avoid warped fungus but like crimson nylium.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   if (HoglinAi.isPosNearNearestRepellent(this, $$0)) {
      return -1.0F;
   } else {
      return $$1.getBlockState($$0.below()).is(Blocks.CRIMSON_NYLIUM) ? 10.0F : 0.0F;
   }
}
```

Striders like lava and don't want to go anywhere else.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   if ($$1.getBlockState($$0).getFluidState().is(FluidTags.LAVA)) {
      return 10.0F;
   } else {
      return this.isInLava() ? Float.NEGATIVE_INFINITY : 0.0F;
   }
}
```

Silverfish like blocks they can infest.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   return InfestedBlock.isCompatibleHostBlock($$1.getBlockState($$0.below())) ? 10.0F :
super.getWalkTargetValue($$0, $$1);
}
```

Guardians like water and prefer higher light levels.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   return $$1.getFluidState($$0).is(FluidTags.WATER) ? 10.0F +
$$1.getPathfindingCostFromLightLevels($$0) : super.getWalkTargetValue($$0, $$1);
}
```

Mooshrooms like mycelium and well-lit blocks.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   return $$1.getBlockState($$0.below()).is(Blocks.MYCELIUM) ? 10.0F :
$$1.getPathfindingCostFromLightLevels($$0);
}
```

Bees, when wandering, prefer air over other non-solids.

```
public float getWalkTargetValue(BlockPos $$0, LevelReader $$1) {
   return $$1.getBlockState($$0).isAir() ? 10.0F : 0.0F;
}
```

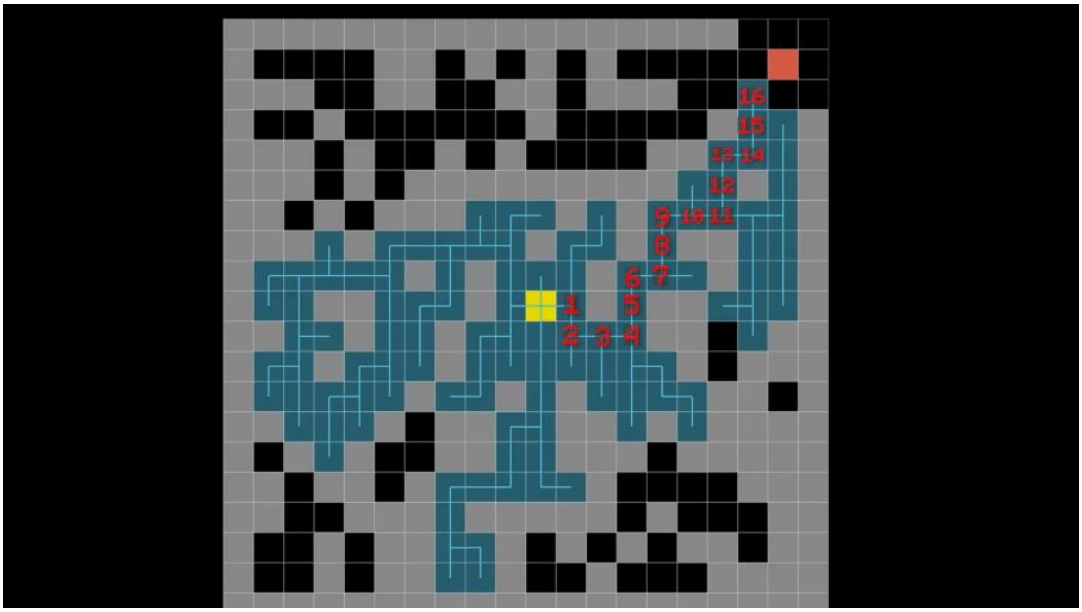
PATHFINDING ALGORITHM

Now that you've seen how this works, you're probably already thinking of ways to mess with it. But before you run off to test things, you need one more tool.

Minecraft mobs use **A*** pathfinding to navigate. This means once the mob chooses a block to move toward, it generates the optimal path to get there.

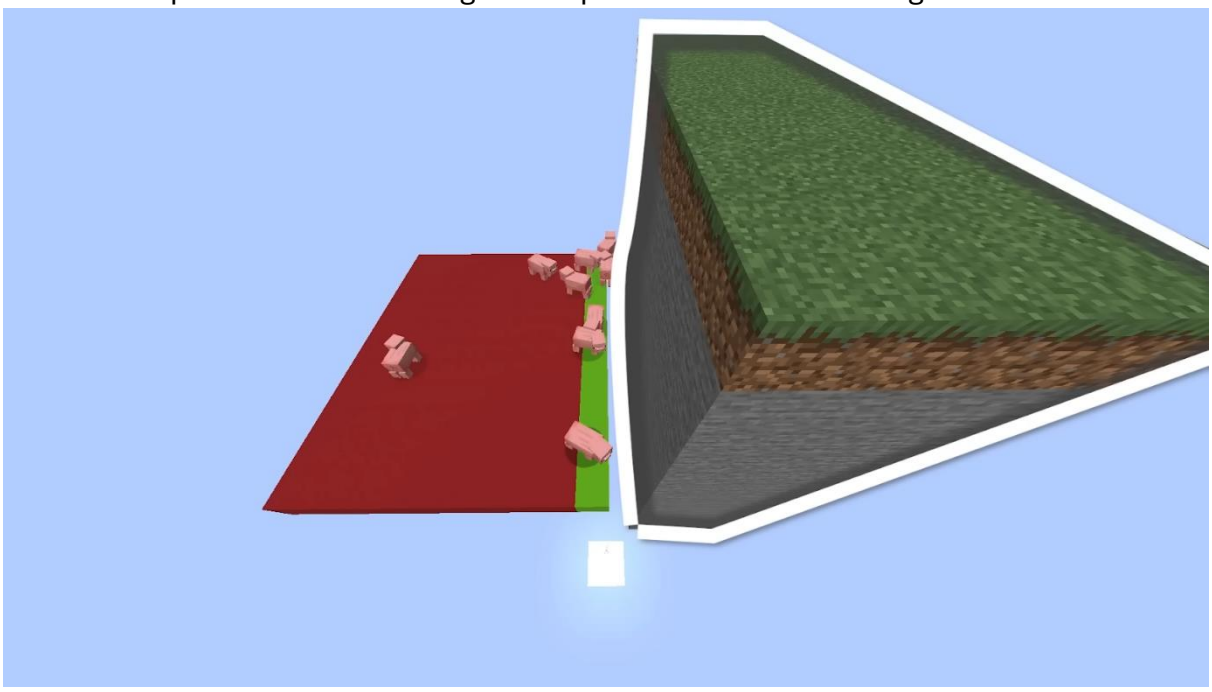
How it does this is more complicated, but you need to know two things:

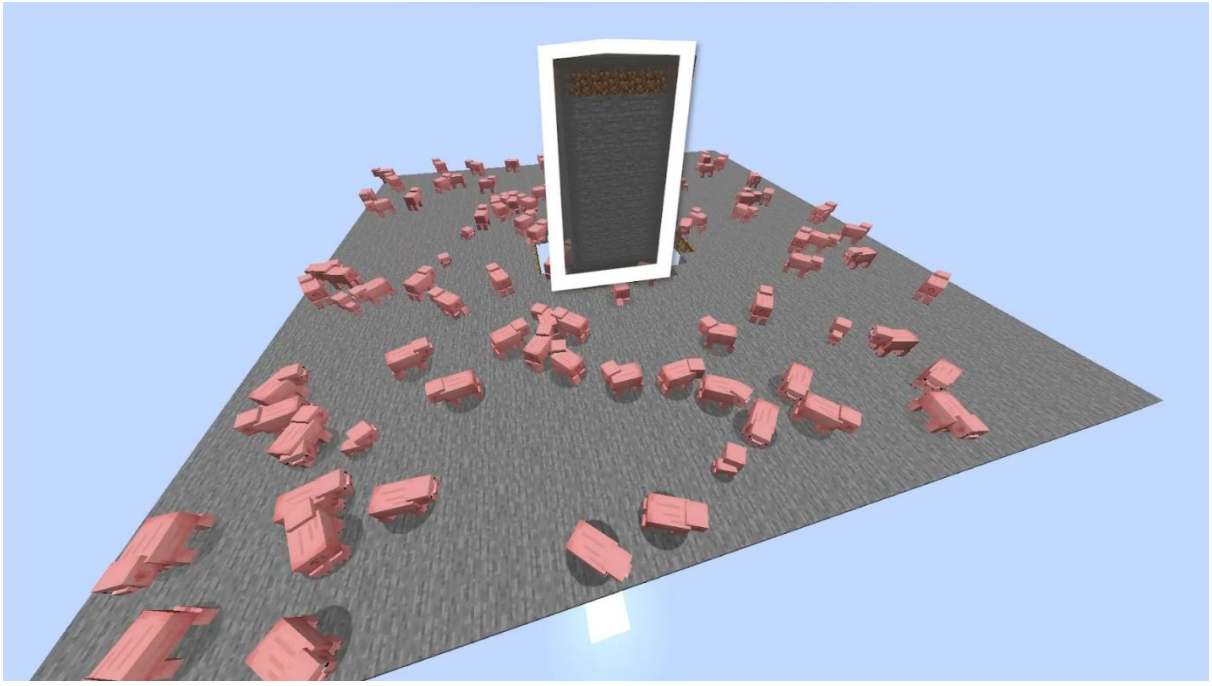
1. A mob will always choose the most optimal path to its destination.
2. A mob will only pathfind up to several blocks equal to its **follow range** - 16 by default for most mobs.



The cool part is that if a mob can't reach the target block within 16 blocks, it will instead pathfind to the **closest block it *can* reach**. This means you can create monoliths that

mobs attempt to reach even though it’s impossible within their range.





Since mobs always choose the closest reachable block, their movement becomes predictable. In the example shown, the pig almost always chooses the top path.

This opens a whole new world: you don’t even need a direct path to the target. A simple stone pillar can cause all pigs to run toward the center of a platform - insanely powerful for farm designs.

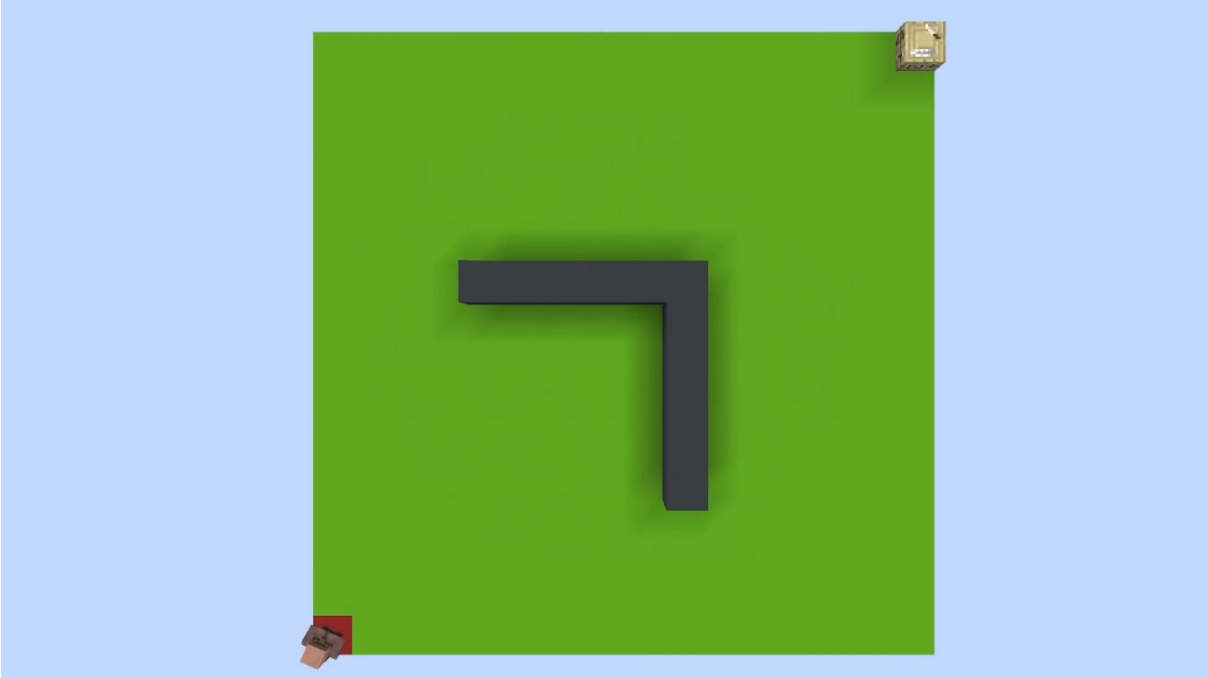
PATHFINDING EXPLOITS

By digging through Minecraft's source code, I've uncovered pathfinding exploits that turn one of the game's most frustrating problems - moving mobs - into something you can completely control. I've read the code for every mob and put together some of the most useful exploits you can use in survival. These range from forcing mobs to take specific paths to making monsters completely unwilling to enter your base. By the end of this guide, you'll understand how mobs think, allowing you to predict and manipulate their behavior in ways most players don't even know are possible.

Before we look at individual mobs, we need to understand the algorithm that governs all of them: **A*** (A-star).

How A* Works in Minecraft

Imagine a villager standing in the bottom-left corner trying to reach a fletching table in the top-right corner. There's a barrier in between, but the villager doesn't know that yet. Its AI knows where the target is, but it still needs to find a path.



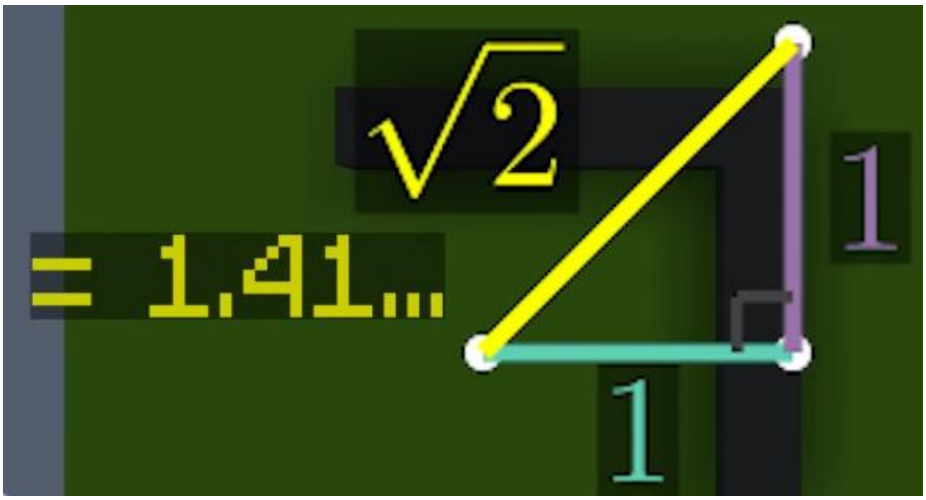
Starting from the villager, the algorithm looks at all adjacent valid spaces and evaluates each one using this formula:

$$min\ f(n) = g(n) + 1.5h(n)$$

It chooses the node with the *smallest sum* of these two values:

g **- Cost so far** - how far the mob has already traveled.

- Moving one block normally costs **1**.
- Moving diagonally costs **~1.41**.

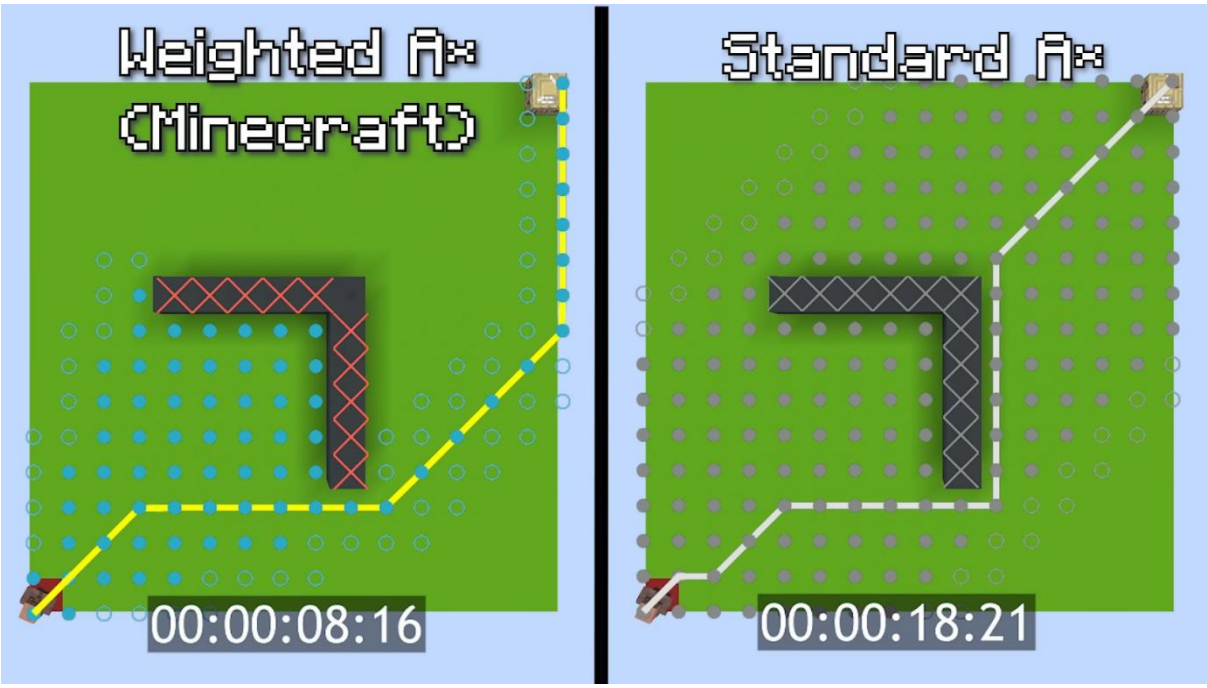


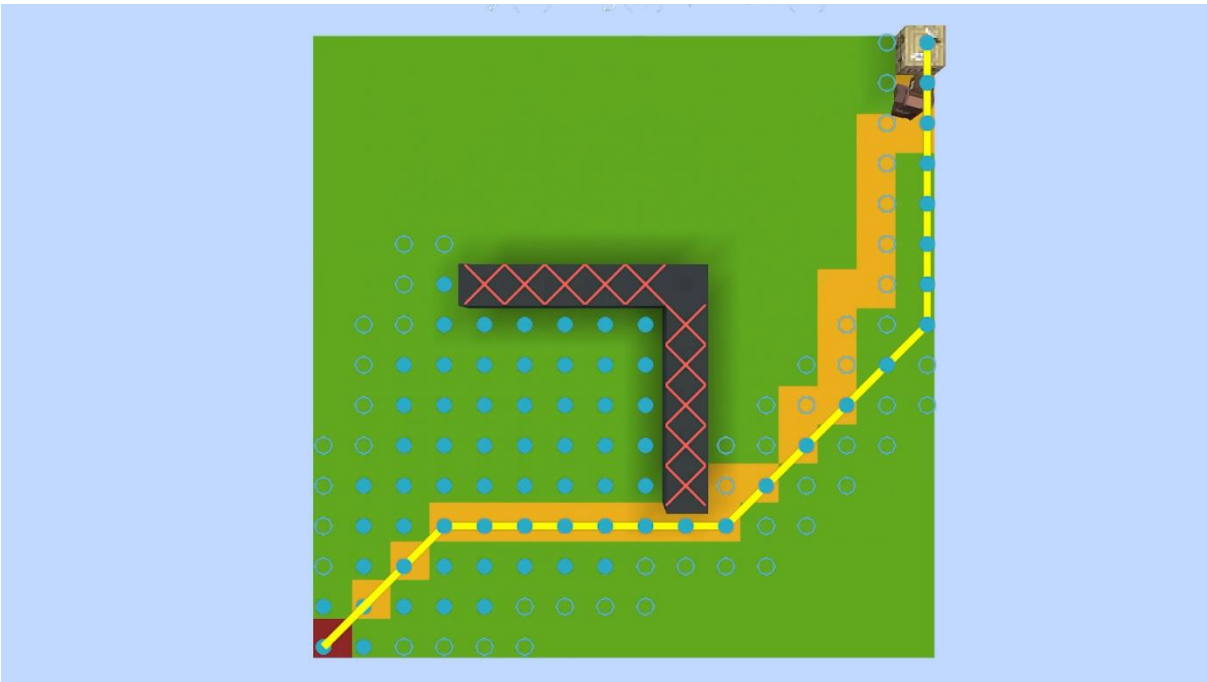
h **- Heuristic** - a guess of how far the mob is from the target.
In Minecraft, this is the straight-line distance to the destination.

Mojang decided to multiply the heuristic by **1.5**, which makes the algorithm faster but not always perfectly optimal. It's still very close to the shortest path, just slightly biased toward moving directly toward the goal.

Once the algorithm finds a path, the mob starts walking along the chosen nodes.

Here is the difference between Standard and Minecraft A*:





The trajectory took by villager 1

Two Major Quirks That Make A* Abusable

1. Block updates force mobs to recalculate their path

Whenever a block updates near the mob's current path, the mob reruns A* with a **1-second cooldown**.

```
public boolean shouldRecomputePath(BlockPos $$0) {
   if (this.hasDelayedRecomputation) {
      return false;
   } else if (this.path != null && !this.path.isDone() && this.path.getNodeCount() != 0) {
      Node $$1 = this.path.getEndNode();
      Vec3 $$2 = new Vec3(((double)$$1.x + this.mob.getX()) / 2.0, ((double)$$1.y + this.mob.getY()) /
2.0, ((double)$$1.z + this.mob.getZ()) / 2.0);
      return $$0.closerToCenterThan($$2, (double)(this.path.getNodeCount() -
this.path.getNextNodeIndex()));
   } else {
      return false;
   }
}
```

This means:

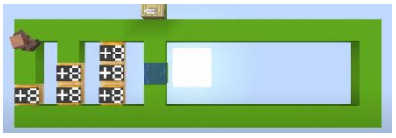
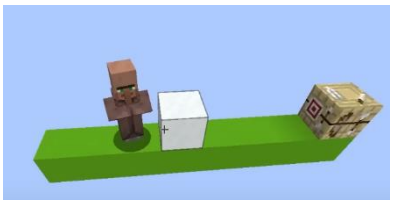
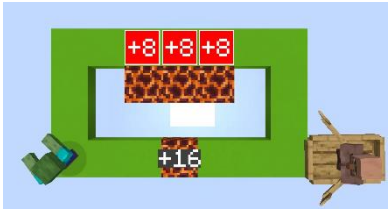
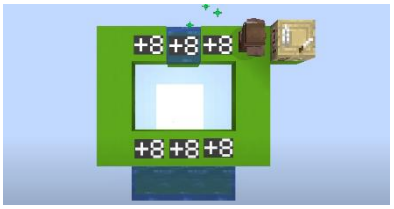
- You can use 1-second block clocks to keep a mob constantly reconsidering its path.
- Large block updates near many mobs cause **huge lag**, because every mob tries to pathfind at once.

This can be abused… but also accidentally triggered.

2. Pathfinding Malice (block cost penalties)

Some blocks cost *extra* to move through or even stand next to, and mobs will avoid them if possible.

Here's a quick reference list:

Block / Condition	Malice / Behavior	Image
Honey block	**+8** cost	
Powder snow	Impassable	
Closed doors	Impassable	
Fire	**+16** to traverse, **+8** for adjacent	
Animals & Villagers	Treat fire as **- 1** (impassable) Fire = **16** (not - 1)	
Cactus / Sweet berry bushes	Impassable; adjacent = **+8**	
Water	**+8** to walk through or next to	

By strategically placing blocks with malice, you can force mobs to take the path *you* want.

The complete list is defined by those enumerations:

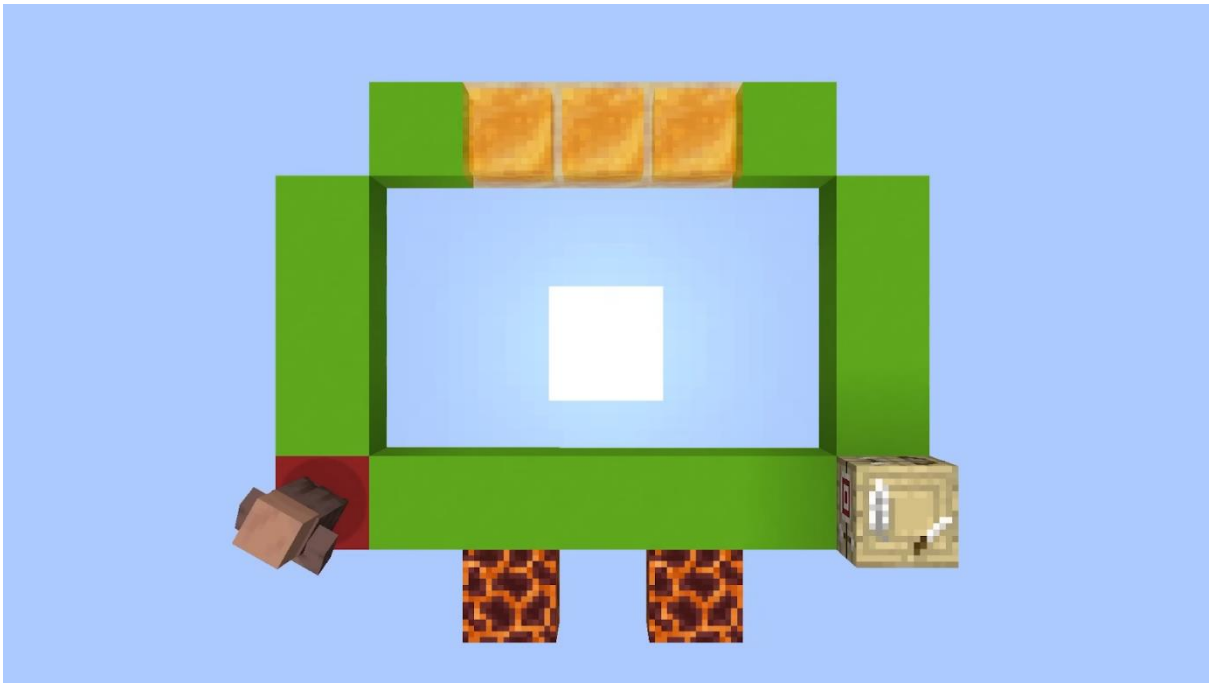
```
1.0F means the block is impassible
0.0F means the block is passible and does not have any maluses
X.0F means the block costs X to pass through
```

```
BLOCKED(-1.0F),
OPEN(0.0F),
WALKABLE(0.0F),
WALKABLE_DOOR(0.0F),
TRAPDOOR(0.0F),
POWDER_SNOW(-1.0F),
DANGER_POWDER_SNOW(0.0F),
FENCE(-1.0F),
LAVA(-1.0F),
WATER(8.0F),
WATER_BORDER(8.0F),
RAIL(0.0F),
UNPASSABLE_RAIL(-1.0F),
DANGER_FIRE(8.0F),
DAMAGE_FIRE(16.0F),
DANGER_OTHER(8.0F),
DAMAGE_OTHER(-1.0F),
DOOR_OPEN(0.0F),
DOOR_WOOD_CLOSED(-1.0F),
DOOR_IRON_CLOSED(-1.0F),
BREACH(4.0F),
LEAVES(-1.0F),
STICKY_HONEY(8.0F),
COCOA(0.0F),
DAMAGE_CAUTIOUS(0.0F),
DANGER_TRAPDOOR(0.0F);
```

BLOCKED is the blocks that obstruct paths
OPEN are the opened blocks
WALKABLE means the block is walkable through
WALKABLE_DOOR are the doors, no matter their state
TRAPDOOR are walkable, no matter their state
POWDER_SNOW are of course impassible
DANGER_POWDER_SNOW means we can go aside a powder snow block without cost
FENCE is considered impassible by mobs
LAVA is considered impassible
WATER means going through water costs 8
WATER_BORDER means walking along water costs 8
RAIL are passibles
UNPASSABLE_RAIL are impassible
DANGER_FIRE means passing ALONG fire cost s 8
DAMAGE_FIRE means passing IN fire costs 16
DANGER_OTHER means passing ALONG costs 8
DAMAGE_OTHER blocks are impassible
DOOR_OPEN is passible
DOOR_WOOD_CLOSED are impassible for mobs who can't open it
DOOR_IRON_CLOSED are impassible
BREACH type blocks costs 4
LEAVES type blocks are impassible
STICKEY_HONEY means going on Honey Block costs 8
COCOA blocks are passible
DAMAGE_CAUTIOUS costs 0, unused enum
DAMAGE_DANGER_TRAPDOOR costs 0, unused enum

```
protected Animal(EntityType<? extends Animal> $$0, Level $$1) {
    super($$0, $$1);
    this.setPathfindingMalus(PathType.DANGER_FIRE, 16.0F);
    this.setPathfindingMalus(PathType.DAMAGE_FIRE, -1.0F);
}
// Some entity redefines maluses. For example, Animal.java redefines DANGER_FIRE and DAMAGE_FIRE
```

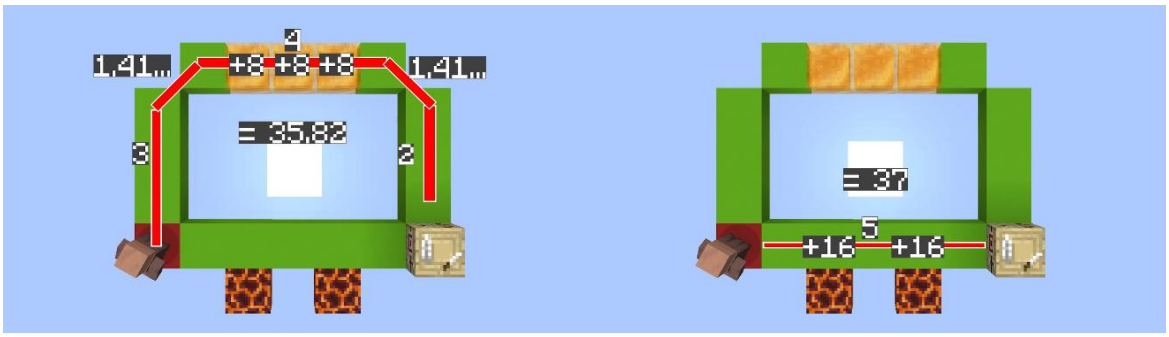
Example: Choosing Between Two Paths



A villager can choose between:

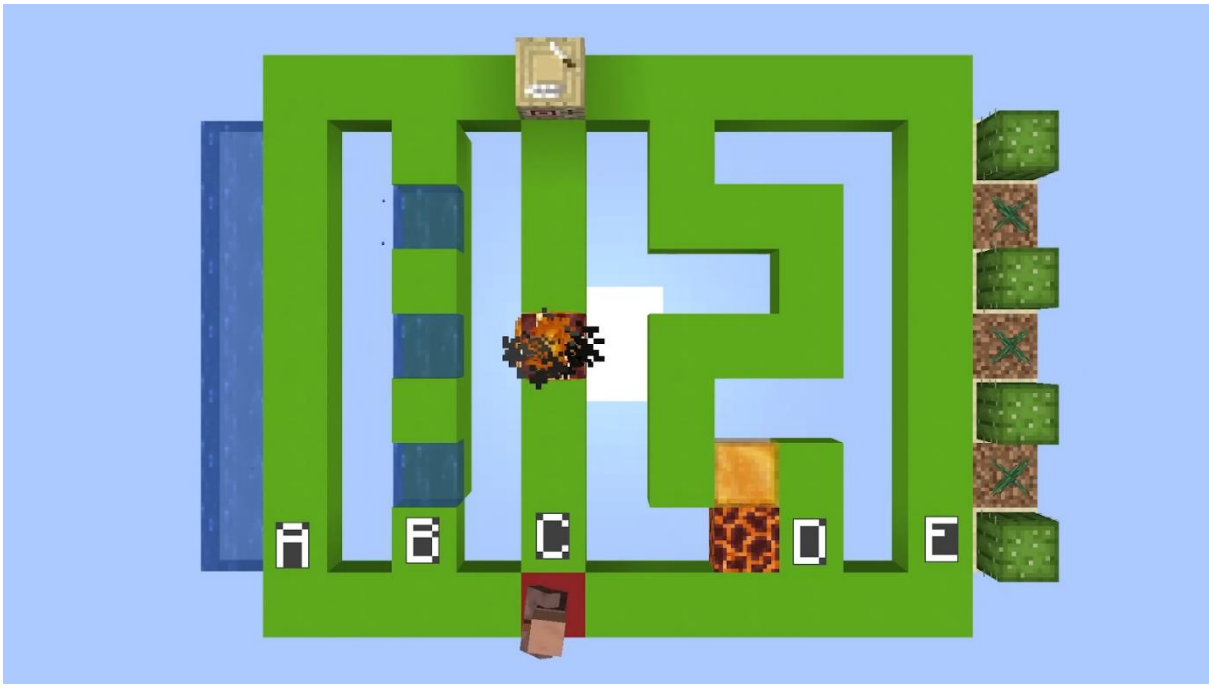
- A short path with **two adjacent magma blocks**
- A longer path with **three honey blocks**

Compute their A* costs and the villager will slightly prefer the magma-adjacent path - because despite the danger, it has a lower total cost.

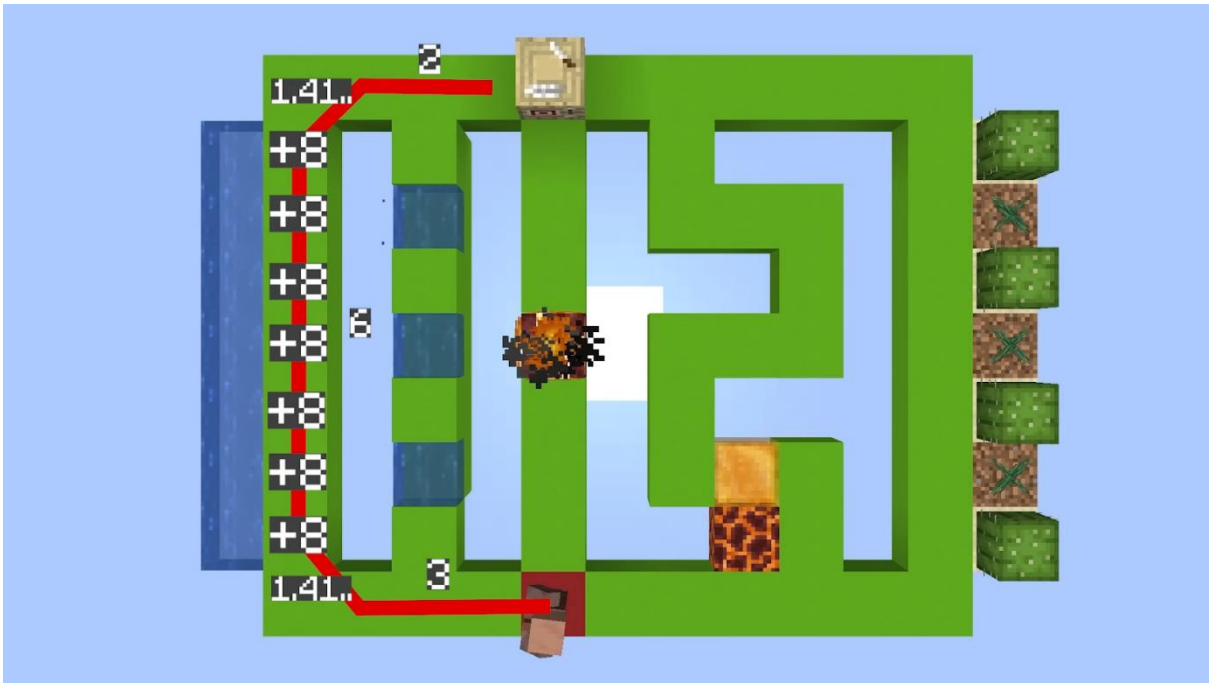


EXERCISE

Look at this puzzle:

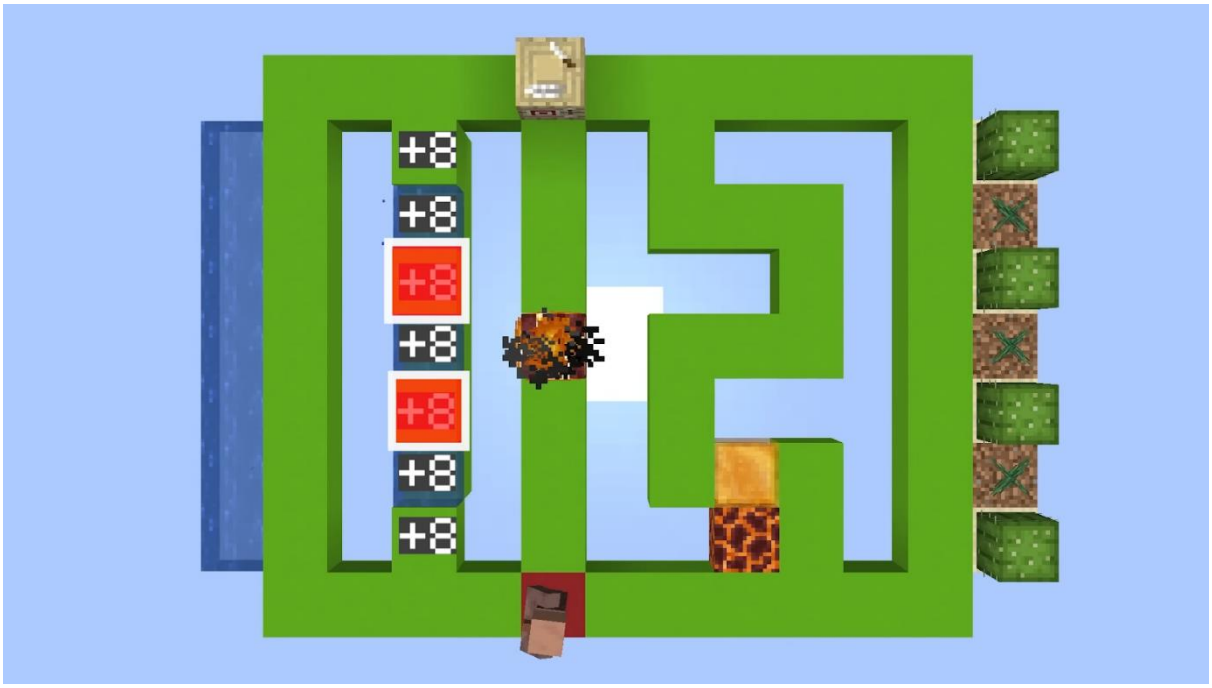


At first, path A may be tempting due to the lack of obstacles but remember that walking next to water has a malice value of eight. If we add all of those up and then include the total distance the villager needs to travel, we can calculate the total cost of this path to be **69.82**.

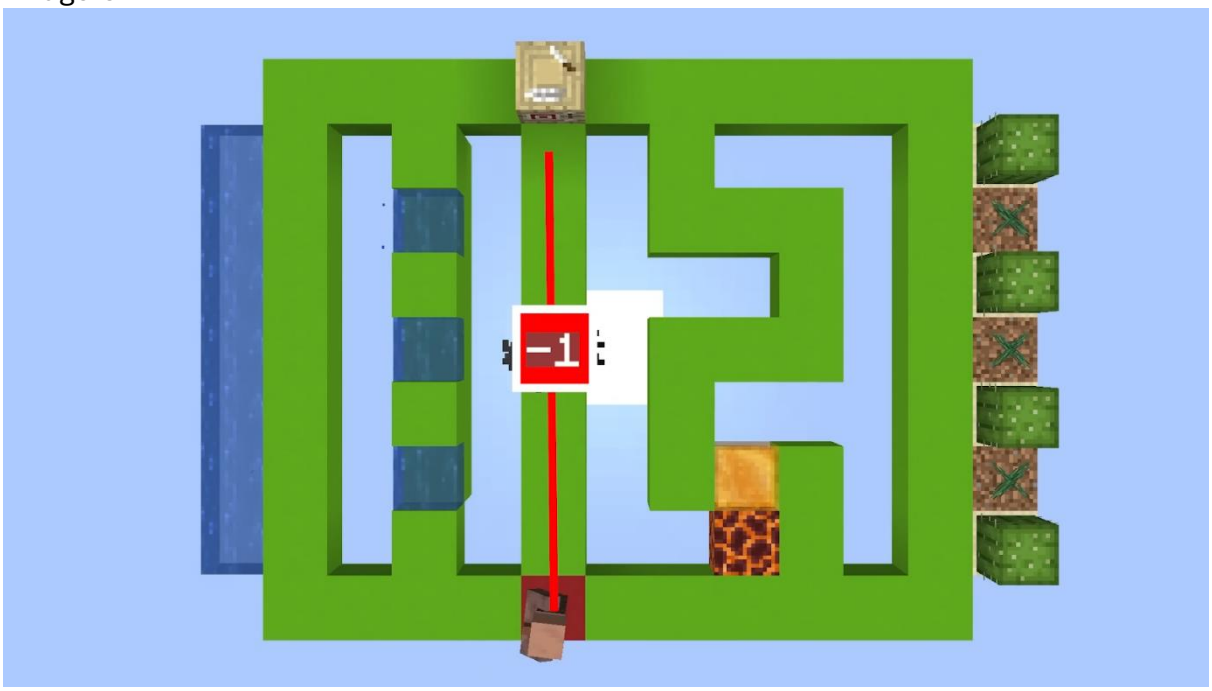


We can do the same thing for the second path, remembering that water also has a malice of eight. Keep in mind that a single malice can't be double counted: these blocks aren't doubly next to water, but a block can have multiple different malice.

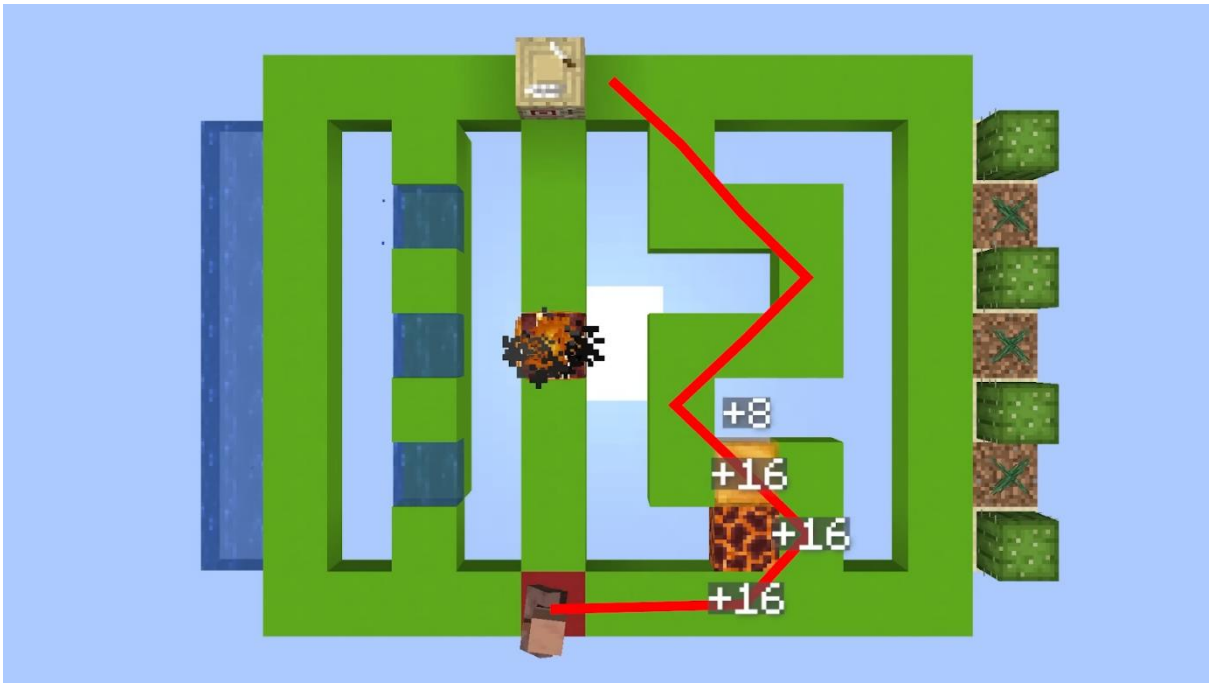
The total for this path is **65.82**.



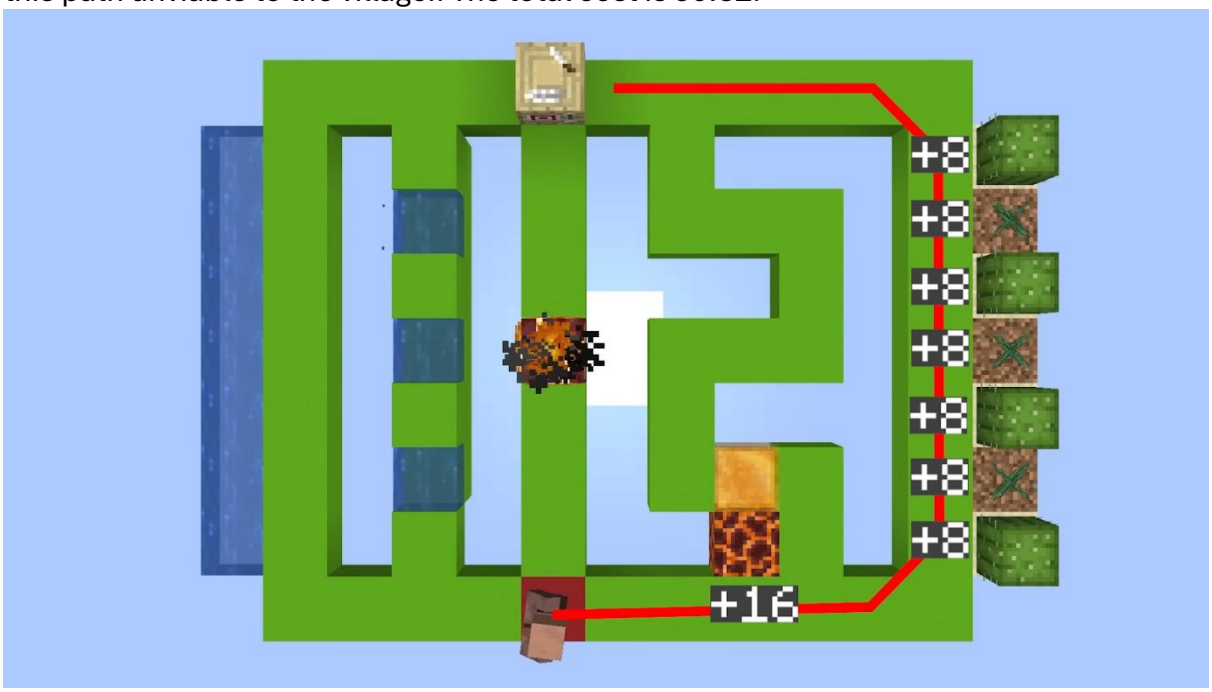
The middle path can be ignored in this case because, as we know, fire is impossible to villagers.



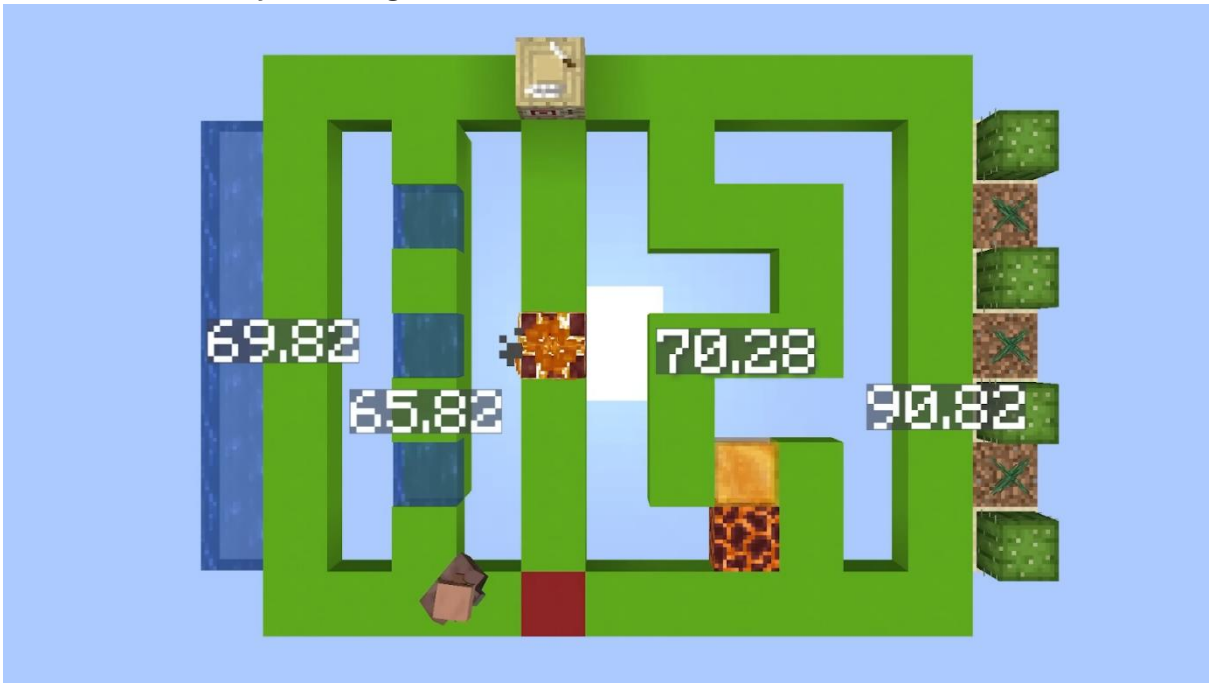
The fourth path has far fewer malice blocks. However, this magma block is hitting the path in three places. And this honey block gets added on top of the malice from the magma block, bringing the total up to **70.82**:



For the last path, the cacti and sweet berry bushes all apply a malice of eight, making this path unviable to the villager. The total cost is **90.82**.



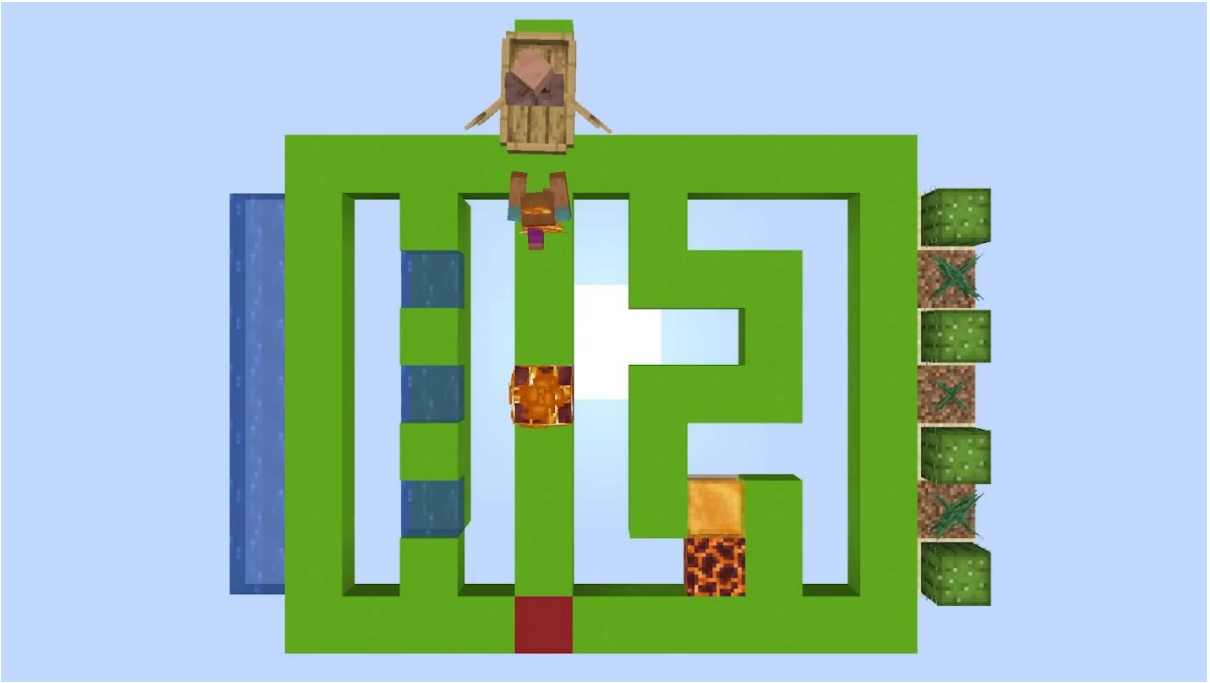
Now that we've calculated all the costs, we can see that the lowest one — path B — is the one selected by the villager.



Different Mobs Choose Different Paths

For instance:

- Villagers treat fire as impassable.
- Zombies do **not**, so they may choose a path villagers refuse to take.



This means different mob types can be manipulated differently using the same layout.

How to Assign Targets to Mobs

Understanding A* is only half the battle. You also need to know how to *give* mobs a target.

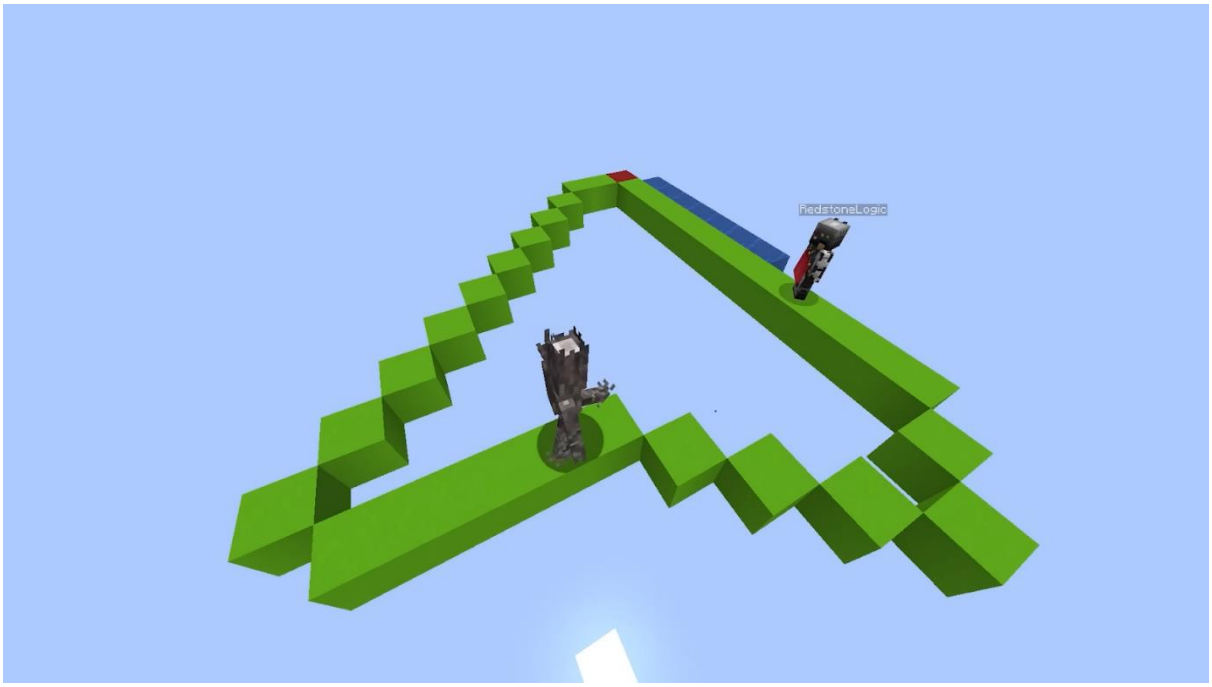
Hostile mobs

They pathfind directly toward:

- The player they detect
- Any valid attack target
- Specific triggers (wardens, piglins, etc.)

Because of this, you can easily force zombies, skeletons, or creepers into traps by manipulating malice blocks.

Creakings, for example, can be made to take absurd detours simply by placing water near the direct route.

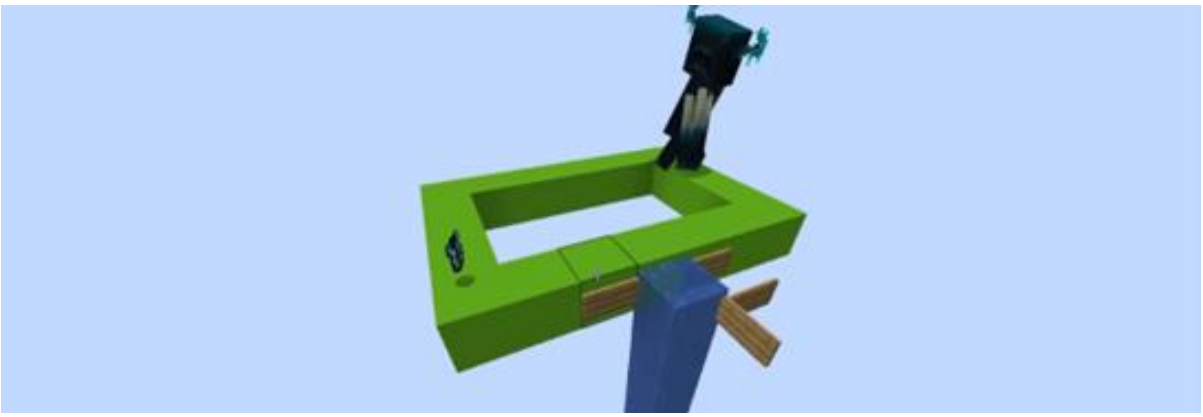


Wardens

Wardens pathfind toward:

- Vibrations
- Players, once their anger is high enough

Making noise repeatedly will switch the warden from “vibration-targeting” to “player-targeting.”



```
public void onReceiveVibration(ServerLevel $$0, BlockPos $$1, Holder<GameEvent> $$2, @Nullable Entity
$$3, @Nullable Entity $$4, float $$5) {
   if (!Warden.this.isDeadOrDying()) {
      Warden.this.brain.setMemoryWithExpiry(MemoryModuleType.VIBRATION_COOLDOWN, Unit.INSTANCE, 40L);
      $$0.broadcastEntityEvent(Warden.this, (byte)61);
      Warden.this.playSound(SoundEvents.WARDEN_TENDRIL_CLICKS, 5.0F, Warden.this.getVoicePitch());
      BlockPos $$6 = $$1;
      if ($$4 != null) {
         if (Warden.this.closerThan($$4, 30.0)) {
            if (Warden.this.getBrain().hasMemoryValue(MemoryModuleType.RECENT_PROJECTILE)) {
               if (Warden.this.canTargetEntity($$4)) {
                  $$6 = $$4.blockPosition();
               }

               Warden.this.increaseAngerAt($$4);
            } else {
               Warden.this.increaseAngerAt($$4, 10, true);
            }
         }

         Warden.this.getBrain().setMemoryWithExpiry(MemoryModuleType.RECENT_PROJECTILE, Unit.INSTANCE,
100L);
      } else {
         Warden.this.increaseAngerAt($$3);
      }

      if (!Warden.this.getAngerLevel().isAngry()) {
         Optional<LivingEntity> $$7 = Warden.this.angerManagement.getActiveEntity();
         if ($$4 != null || $$7.isEmpty() || $$7.get() == $$3) {
            WardenAi.setDisturbanceLocation(Warden.this, $$6);
         }
      }
   }
}
```

Animals

Easiest method: **set them on fire**.
They will run directly into water.

Food also works, but fire is more predictable for path manipulation.

```
protected void registerGoals() {
   this.goalSelector.addGoal(0, new FloatGoal(this));

   // Because PanicGoal is lower than TemptGoal,
   this.goalSelector.addGoal(1, new PanicGoal(this, 1.25));
   this.goalSelector.addGoal(4, new TemptGoal(this, 1.2, $$0 -> $$0.is(Items.CARROT_ON_A_STICK),
false));
   this.goalSelector.addGoal(4, new TemptGoal(this, 1.2, $$0 -> $$0.is(ItemTags.PIG_FOOD), false));

   this.goalSelector.addGoal(3, new BreedGoal(this, 1.0));
   this.goalSelector.addGoal(5, new FollowParentGoal(this, 1.1));
   this.goalSelector.addGoal(6, new WaterAvoidingRandomStrollGoal(this, 1.0));
   this.goalSelector.addGoal(7, new LookAtPlayerGoal(this, Player.class, 6.0F));
   this.goalSelector.addGoal(8, new RandomLookAroundGoal(this));
}
```

Villagers

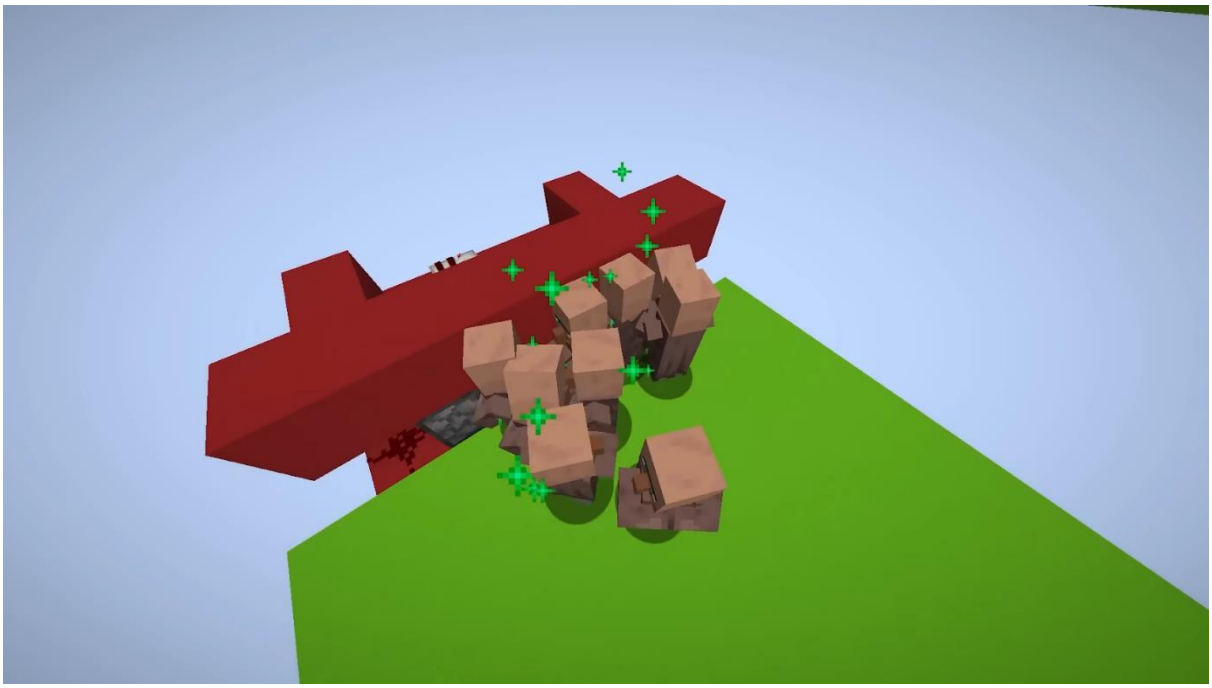
Villagers pathfind to:

- **Unclaimed workstations** from up to **48 blocks away**

```
Set<Pair<Holder<PoiType>, BlockPos>> $$12 = (Set)$$10xx.findAllClosestFirstWithType(
   $$0, $$11, $$8x.blockPosition(), 48, PoiManager.Occupancy.HAS_SPACE
)
```

- Their job block at specific times of day

If you move a workstation rapidly, villagers keep their current path, but the workstation becomes “unclaimed” again, allowing you to gather many villagers into one location.



This enables:

- Villager relocation without minecarts
- Villager grouping systems
- Automatic villager herding farms

Recap:

Credits go to Coolguy1802

Minecraft Pathfinding Cheat Sheet

Mob pathfinding in Minecraft can be triggered by a mob every 1 second, and is re-triggered on nearby block updates. Mobs always choose the path with the lowest cost after running a weighted A* algorithm. The cost of a block that a mob can walk on is based on the values below.

Passable Blocks

+1 - Any "normal" block horizontally or vertically
+1.41 Any "normal" block diagonally

+8 - Honey Block

+8 - Water Block
+8 - Blocks next to water

Impassable Blocks

IMPASSABLE - Cactus

IMPASSABLE - Sweet Berry

IMPASSABLE - Powdered Snow

Fire Blocks

Hostile Mobs
+16 - All fire blocks
+8 - Blocks next to fire

Passive Mobs
IMPASSABLE - All fire blocks
+16 - Blocks next to fire

Inspired by RedLogic videos:

Minecraft's Smartest System Is Almost Completely Untapped
Nobody Realized You Could Mind-Control Mobs Like This... Until Now